Job Description - External

Harness CI/CD Expertise:
Proficiency in setting up and managing Harness CI/CD pipelines for enterprise systems.
Experience in creating CI/CD pipelines across any 2 technologies (Java, .NET, Lambda, Python and Database) with a focus on build, automation and deployment optimization.
Hands-on experience integrating SaaS platforms (e.g., JIRA, JFrog, DataFlower, FS) and hosted platforms (e.g., GitHub, Jenkins, SonarQube, Checkmarx) into Harness pipelines.
Knowledge of deployment patterns such as Blue-Green and Canary.
Familiarity with quality gates (e.g., SonarQube, Checkmarx) and artifact repositories like JFrog Artifactory.
Expertise in configuring Harness Delegate Provisioning, RBAC rules and LDAP Integrations setups.
Cloud and Infrastructure as Code (IaC):
Strong expertise in AWS and cloud infrastructure management.
4years of experience with Terraform for provisioning across development, staging, and production environments.
CI/CD Tools and Pipelines:
Experience with Jenkins, Harness or similar CI/CD tools for pipeline development
Proficiency in pipeline scripting, including Jenkins and Harness-specific pipelines.
Version Control:
Expertise in GitHub, including enterprise-level setups and integration with CI/CD pipelines.
Programming and Scripting:
4years of experience with scripting languages such as Python, Shell and Terraform for automation and troubleshooting.
Containers and Orchestration:
1years of hands-on experience with Docker and Kubernetes (CKA certification preferred).
Knowledge of container image repositories such as ECR.
Secondary Skills
Migration and Integration Support:
Hands on experience with pipeline migration strategies from Jenkins or other CI/CD tools to Harness.
Familiarity with integrating SSO, SailPoint, Service Now, Jira, SonarQube and other enterprise tools into the Harness ecosystem.
Configuration and Automation:
Experience customizing accounts with roles, policies, OPA and security groups using Terraform and other automation tools.
Observability and Monitoring Tools:
Knowledge of monitoring and reporting mechanisms for CI/CD pipeline performance and reliability.
Data and Messaging Tools:
Familiarity with Kafka, Zookeeper, and ELK Stack for application monitoring and data flow optimization.

i need interview questions & answers

Here are some **scenario-based interview questions** along with suggested answers for the **Harness CI/CD, AWS, Terraform, Kubernetes, and DevOps-related role**:

Harness CI/CD

1. **How do you design a CI/CD pipeline in Harness for deploying a Java-based microservices application?**
   - **Answer:**
     - First, define a **CI stage** that builds the Java application using Maven or Gradle.
     - Integrate GitHub/Jenkins for source control and trigger builds on code commits.
     - Use **SonarQube and Checkmarx** for quality and security scans.
     - Store the generated artifact in **JFrog Artifactory** or an S3 bucket.
     - In the **CD stage**, configure Kubernetes or ECS for deployments.
     - Implement **Blue-Green** or **Canary deployments** for safer rollouts.
     - Use Harness **RBAC and Delegate Provisioning** to ensure secure access.
2. **How would you migrate a Jenkins-based CI/CD pipeline to Harness?**
   - **Answer:**
     - Analyze the existing **Jenkins jobs** (e.g., build, test, deploy) and translate them into Harness **pipeline stages**.
     - Replace Jenkins pipeline scripting with Harness **YAML-based configuration**.
     - Migrate integrations (e.g., GitHub, JIRA, SonarQube, Checkmarx) to Harness connectors.
     - Move artifact storage from Jenkins to Harness-supported platforms like **JFrog or AWS S3**.
     - Implement **RBAC policies** and **LDAP authentication** for secure access.
3. **Explain how Blue-Green and Canary deployments work in Harness.**
   - **Answer:**
     - **Blue-Green Deployment:**
       - Two identical environments (**blue and green**) are maintained.
       - The new version is deployed to **Green**, tested, and then traffic is switched from **Blue** to **Green**.
       - If issues arise, rollback is instant by switching traffic back to **Blue**.
     - **Canary Deployment:**
       - A small percentage of traffic is sent to the **new version** while monitoring metrics.
       - If successful, more traffic is gradually shifted until full rollout.
       - If failures occur, rollback is easier since only a portion of users are affected.

Terraform & AWS

4. **How do you structure Terraform modules for a scalable AWS deployment?**
   - **Answer:**
     - Break down infrastructure into reusable **modules** (e.g., VPC, EC2, RDS, IAM).
     - Use a **root module** to call sub-modules with appropriate **variables and outputs**.
     - Maintain state using **remote backends** like S3 + DynamoDB for locking.
     - Implement **Terraform Workspaces** for managing different environments.
     - Use Terraform **best practices** such as `terraform fmt`, `plan`, and `validate`.
5. **How do you ensure Terraform infrastructure changes do not break production?**
   - **Answer:**
     - Use **Terraform Plan** to preview changes before applying them.
     - Implement **GitHub Actions/Jenkins/Harness pipelines** for Terraform execution.
     - Enable **Terraform state locking** using DynamoDB to prevent conflicts.
     - Deploy infrastructure in **stages** (e.g., dev → staging → prod).
     - Use **Feature Flags** to toggle changes safely.
6. **How do you handle IAM permissions in AWS using Terraform?**
   - **Answer:**
     - Use Terraform's **IAM module** to define roles, policies, and permissions.
     - Implement **least privilege access** using AWS-managed policies.
     - Use Terraform **dynamic blocks** to generate fine-grained access rules.
     - Store sensitive IAM credentials securely using **AWS Secrets Manager**.

Kubernetes & Docker

7. **How do you deploy a microservices application using Kubernetes in Harness?**
   - **Answer:**
     - Use Harness **Kubernetes Connector** to integrate with the cluster.
     - Define **manifests in Helm or Kustomize** and store them in a Git repository.
     - Use Harness **GitOps** for deploying and syncing Kubernetes configurations.
     - Implement **horizontal pod autoscaling (HPA)** and **resource limits**.
     - Set up **canary deployments** using traffic routing with Istio/NGINX.
8. **How would you troubleshoot a failed deployment in Kubernetes?**
   - **Answer:**
     - Check **pod logs** using `kubectl logs <pod_name>`.
     - Inspect pod events using `kubectl describe pod <pod_name>`.
     - Verify if the container is restarting using `kubectl get pods -w<watch>`.
     - Check network issues using `kubectl get services` and `kubectl get ingress`.
     - Look at Helm or Kustomize errors in the deployment pipeline.
9. **How do you secure container images in a CI/CD pipeline?**
   - **Answer:**
     - Use **JFrog Xray or Clair** to scan images for vulnerabilities.
     - Store images in **private repositories** like ECR or JFrog Artifactory.
     - Sign images using **Cosign or Notary** to ensure integrity.
     - Implement **role-based access (RBAC)** for pulling/pushing images.

CI/CD Tools & Integrations

10. **How do you integrate SonarQube and Checkmarx into Harness for security scanning?**

- **Answer:**
  - Configure SonarQube and Checkmarx as **Quality Gates** in the pipeline.
  - Add a **scan step** after the build phase to check for vulnerabilities.
  - Fail the pipeline if security thresholds (e.g., OWASP Top 10) are exceeded.
  - Use **Checkmarx SAST/DAST** for in-depth security analysis.

11. **How do you set up Harness Delegate Provisioning for secure deployment?**

- **Answer:**
  - Install Harness Delegate on a **private VPC** or Kubernetes cluster.
  - Assign **least privilege IAM roles** to the Delegate.
  - Use **auto-scaling** to handle workload variations.
  - Monitor delegate health via **Harness UI and logs**.

12. **How do you integrate Jira and ServiceNow into a Harness pipeline?**

- **Answer:**
  - Use Harness **Jira Connector** to create/update tickets automatically.
  - Implement **approval gates** where a Jira ticket must be resolved before deployment.
  - Integrate **ServiceNow APIs** to trigger ITSM workflows post-deployment.

Observability & Monitoring

13. **How do you monitor Harness CI/CD pipelines for failures?**

- **Answer:**
  - Use **Harness built-in logs and dashboards**.
  - Forward logs to **Splunk, ELK Stack, or Datadog** for centralized monitoring.
  - Set up **alerts in Prometheus/Grafana** for critical failures.

14. **How would you use Kafka in a CI/CD environment?**

- **Answer:**
  - Kafka can be used for **event-driven CI/CD pipelines**.
  - It can **stream deployment logs** to an ELK or Splunk system for real-time monitoring.
  - Use Kafka for **asynchronous job processing** (e.g., triggering automated tests).

These questions should help you **prepare for your interview** with both **technical and scenario-based answers**. Do you need more in-depth answers on any topic? 🚀